The University of Lahore
Department of Computer Science & IT
Final Term Examination

Fall-2023

Course Title:	Computer Organization & Assembly Language	Course Code:	CS11207	Credit Hours:	**4**
Course Instructors:	Mr. M. Yasin, Ms. Asmara, Ms.Uzma, Mr.Mutahher, Ms.Zarbakht, Mr.Asif	Program Name:	**BSCS**		
Time Allowed:	**120 Minutes**	**Maximum Marks:**	**40**		
Date:	**04-01-24**	Course Mentor:	**Mr. M. Yasin Nasir**		
Student's Name:		Signature:			
Reg. No:		Section:	**A,B,C,D,E,F,G,I,J**		

Question # 1: Find the output of the following questions. (5 marks)

```
.MODEL SMALL
.STACK 100H
.DATA
A DB 1, 2, 3, 4
B DB 6, 7, 8, 9
.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX
MOV CX, 0000
MOV CL, 03
LEA BP, A
LEA SI, B
L1:
MOV DH, [BP]
MOV DL, [SI]
MOV [BP], DL
MOV [SI], DH
MOV DH, [BP]
MOV DL, [SI]
MOV DL,DH
ADD DL,'0'
MOV AH,02
INT 21h
INC BP
INC SI
DEC CL
CMP CL, 00
JNZ L1
MOV AH, 4CH
INT 21H
END MAIN
```

Output:

```
K:\>EXAM01.EXE
678
K:\>_
```

Question # 2: Dry run the following code for the given values as decimal input. (Note: INDEC & OUTDEC procedure returns decimal input in AX. You don't need to write INDEC and OUTDEC procedures) (6 marks)

```
.MODEL SMALL
.STACK 100H

 .DATA
   PROMPT_1  DB  'Enter a number : $'
   PROMPT_2  DB  0DH,0AH,'Resultant: $'

 .CODE
   MAIN PROC
     MOV AX, @DATA
     MOV DS, AX
     LEA DX, PROMPT_1
     MOV AH, 9
     INT 21H
     CALL INDEC
     CALL UNKNOWN
     PUSH AX
     LEA DX, PROMPT_2
     MOV AH, 9
     INT 21H
     POP AX
     CALL OUTDEC
     MOV AH, 4CH
     INT 21H
   MAIN ENDP

UNKNOWN PROC
   ; Input: AX

   MOV CX, AX
   MOV AX, 1
   @LOOP:
     MUL CX
   LOOP @LOOP
   RET
UNKNOWN ENDP

END MAIN
```

Dry-run for following values:

INDEC Values	Output of the program (OUTDEC)
3	
4	
5	

Space for rough work.

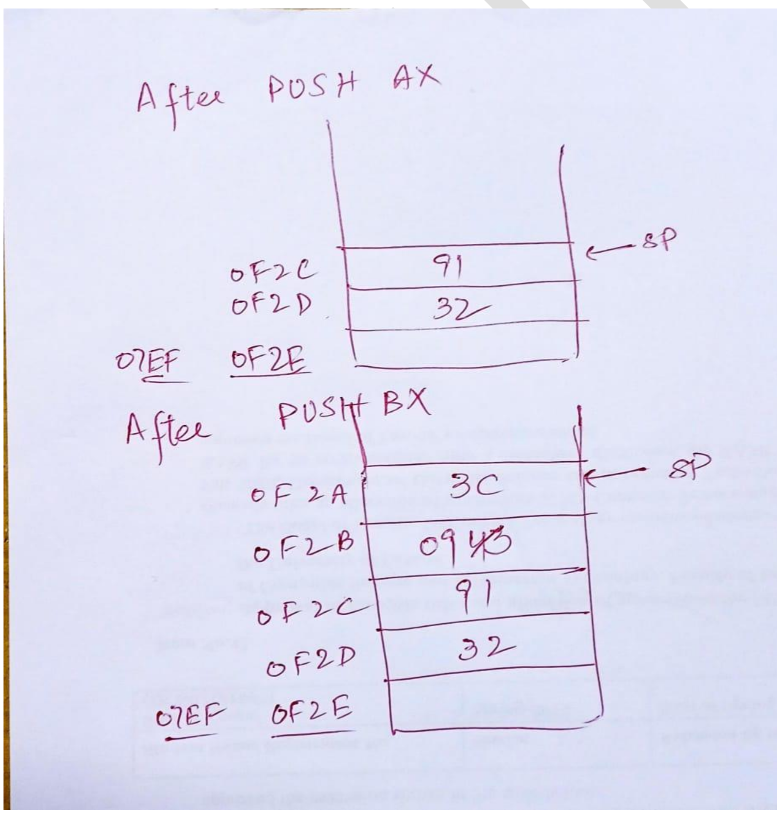
```
K:\>E3FACT~1.EXE
Enter a number: 3

Resultant: 6
K:\>E3FACT~1.EXE
Enter a number: 4

Resultant: 24
K:\>E3FACT~1.EXE
Enter a number: 5

Resultant: 120
```

Question # 3: **(6 Marks)**

Assume that $IP = 100H,\ SS = 07EF$

- **Find the contents of the stack for STACK_OPERATIONS .**
- **Fill the data in given stacks with appropriate Logical Addressing, Stack data values and SP.**
- **Show the sequence of instructions needed at the empty lines in the program to restore all the registers to their original values.**

**.MODEL SMALL
.STACK 0F2Eh
.DATA
.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX
MOV AX,3291H
MOV BX,0F43CH
MOV CX,09H
 	
CALL STACK_OPERATIONS
 	
MOV AH,4CH
INT 21H
MAIN ENDP
END MAIN**
	**STACK_OPERATIONS PROC
	
	PUSH AX
	MOV BH,09H
	PUSH BX
	
	XCHNG CX, AX
	________
	POP BX
	POP AX
	________
	________
	RET
	STACK_OPERATIONS ENDP**



Question # 4: **(3+6 Marks)**

a. Write instructions to copy one STRING into another STRING in reverse order.

b. Write the following microinstructions:

1. **Read memory location addressed by AR and move the data to DR.**
2. **Clear the bit-3 in AL register using mask operations.**
3. **A register AL contains some value. Write instructions to find the 2's complement in AL.**

```
.MODEL SMALL ; (a)
.STACK 100H
.DATA
STRING DB 'keep moving and keep solving',
'$'
.CODE
MAIN PROC FAR
MOV AX,@DATA
MOV DS,AX

CALL REVERSE

LEA DX,STRING
MOV AH, 09H
INT 21H
MOV AH, 4CH
INT 21H
MAIN ENDP

REVERSE PROC
        MOV SI, OFFSET STRING
        MOV CX, 0H
        LOOP1:
        MOV AX, [SI]
        CMP AL, '$'
        JE LABEL1
        PUSH [SI]
        INC SI
        INC CX
        JMP LOOP1
        LABEL1:
        MOV SI, OFFSET STRING
                LOOP2:
                CMP CX,0
                JE EXIT
                POP DX
                XOR DH, DH
                MOV [SI], DX
                INC SI
                DEC CX
                JMP LOOP2

        EXIT:
        MOV [SI],'$ '
        RET
REVERSE ENDP
END MAIN
```

(b)

$$DR \leftarrow MR[AR]$$

$$AL \leftarrow AL \wedge (11110111)$$

$$AL \leftarrow \overline{AL} + 1$$

Question # 5: (6 marks)

Write down a program for the following:

Given: the last two digits of your SAP ID as hexadecimal digits e.g. 701223**12 has 12h.**
Write an assembly language program that consider the given value in BL then complement the bit-3 and bit-4 in BL. Use procedure BINOUT to display the binary value in BL.

Hint: Use mask operation to complement the selected bits.

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
MOV AX,@DATA
MOV DS,AX

MOV BL,12H
XOR BL,18H

CALL OUTBINARY

MOV AH,4CH
INT 21H
MAIN ENDP

OUTBINARY PROC
;input BX
        MOV CX,8
L0:
        ROL BL,1
        JC ONE
        MOV AH,2
        MOV DL,'0'
        INT 21H
        JMP L1
        ONE:
        MOV AH,2
        MOV DL,'1'
        INT 21H
        L1: LOOP L0
RET
OUTBINARY ENDP
END MAIN
```

Question # 6: (8 Marks)

Write an assembly language program where Data Segments contains a string consisting of words separated by blanks, ending with a "$".
Displays the string on a new line but with reversed word. You need to write a procedure that read the word from data section and reverse each word.
For example, "keep calm and keep solving" becomes "peek mlac dna peek gnivlos".

(Note: It can be implemented through stack or string operations. You can use any technique as your convenience)

```
.MODEL SMALL
.STACK 100H

.DATA
                MYVAR DB ?
                REVSTR DW 255 DUP (?)

.CODE
MAIN PROC
                MOV AX,@DATA
                MOV DS,AX
                MOV BX, OFFSET revStr
                MOV SI,0

                CALL REVERSE

EXIT:
                MOV AX,'$'
                MOV [BX][SI],AX
                MOV AH,9
                LEA DX,[REVSTR]
                INT 21H
                MOV AX,4C00h
                INT 21h


                MAIN ENDP

REVERSE PROC

                XOR CX,CX

WHILE_:
                MOV AH,1
                INT 21H

                CMP AL,0DH
                JE REVERSE_
                CMP AL,32
                JE REVERSE_

                PUSH AX
                INC CX

                JMP END_
```

```
REVERSE_:
                MOV MYVAR,AL
                JCXZ EXIT

BOTTOM_:
                POP DX
                MOV [BX][SI],DX
                INC SI
                LOOP BOTTOM_

                MOV DX,32
                MOV [BX][SI],DX
                INC SI
END_:
                CMP MYVAR,0DH
                JE EXIT
                JMP WHILE_


RET
REVERSE ENDP
END MAIN
```